조별 발표 요약

조별 발표 내용 요약

전체 12개 조의 발표 내용을 조별로 정리한 문서입니다.

1조 — AI 에이전트 코스레지스트레이션 어시스턴트 (수강신청 도우미)

주제: 한양대 수강신청을 도와주는 LLM 기반 에이전트 시스템

문제 정의:

- 수강신청 기준이 매학기 머릿속에서 달라짐
- 정보가 수강신청 사이트·포털·에브리타임으로 분산
- 후보 과목 선택이 수동이라 조건 누락·편향 발생
- 시간표 조합 수가 폭발적으로 많음

솔루션: 자연어로 입력하면(예: "전공 학점, 교양 3학점, 월요일 공강") LLM + RAG + 검증도구가 졸업요건과 맞는 시간표를 근거와 함께 추천

핵심 기술:

- LLM: 하드 컨스트레인트(필수)와 소프트 프리퍼런스(선호) 구분
- RAG: 강의계획서·강의평·졸업요건에서 관련 내용만 검색
- Vector DB: 임베딩 의미검색 + 메타데이터 필터링
- 휴리스틱 알고리즘: 사전 필터링 후 시간표 조합

파이프라인: 사용자 쿼리 → RAG 검색 + Structured DB 매칭 → 시간표 생성 → Top-K 랭킹(쿼리 유사도, 리뷰 스코어, 졸업 요건 기여도, 스케줄 선호도) → LLM 설명 생성

아키텍처: AWS 람다 기반 서버리스(EC2 보완), S3(비정형), DynamoDB(정형), CloudWatch 모니터링

팀 구성: 김동호(PM, 아키텍처/LLM·RAG), 정재희·김경호(프론트엔드/백엔드/AWS), 박수진·서지원(쿼리 분석/프롬프트/벡터 DB)

일정: 8주 (요구사항 → 핵심 기능 구현 → 통합 → 테스트)

2조 — Itinerary (AI 여행 일정 플래너)

주제: 개인화된 여행 일정을 자동 생성하는 AI 트래블 어시스턴트

문제 정의:

- 여행 계획은 시간 소모적이며 정보가 여러 플랫폼에 흩어져 있음
- 정보 과부하, 개인화 부족, 시간 경과로 인한 정보 노후
- 지리적 근접성·운영시간·예산·날씨를 동시에 최적화하기 어려움

관련 작업: Wanderlog (여행 일정 앱이지만 개인 맞춤화 부족)

제안 시스템:

- LLM 기반 맞춤화 (솔로/가족/친구 등 목적별)
- 날씨·지도 API 통합
- 호텔·항공편 자동 연동 (실시간 업데이트)
- 실시간 적응형 도구 (자동 재경로)
- 챗봇 인터페이스, PDF 다운로드

아키텍처 (에이전트 6단계):

1. Preference Extraction Agent - 사용자 입력 파싱
2. RAG Agent - 여행 지식 검색
3. Live Data Agent - 실시간 정보(날씨/지도/환율)
4. Planning/Scheduler Agent - 일별 계획 생성
5. Constraint Validator Agent - 현실성 검증
6. Itinerary Generator LLM - 최종 일정 생성

평가: Precision@K, 예산 편차 5% 이내, 사용자 테스트

일정: 6주 (계획 → 백엔드 → 필터링/RAG → 통합 → 평가 → 발표)

발표자: Sal Sabila (대표)

3조 — 에이전틱 페르소나 오케스트레이션 플랫폼

주제: 사용자가 원하는 전문가 페르소나를 즉석 소환하여 토론시키는 플랫폼

팀원: 김민겸, 박준호, 신현성, 이정훈, 조준영

문제 정의:

- 정적 지식과 환각 문제
- 수동적 지식 구축 (즉각 대응 불가)
- 다중 에이전트 환경의 지식 오염

핵심:

1. **즉석 페르소나 생성** - 사전 준비 없이 전문가 그룹 소환

2. **자기 교정형 검색** (Self-Correction RAG)
3. **변증법적 토론 구조** (Multi-Agent Debate)

활용 분야: 교육(역사 인물 토론), 비즈니스(시장 반응 시뮬레이션), 컨텐츠(캐릭터 테스트)

이론적 배경:

- Multi-Agent Debate (MIT 2023, 환각 억제)
- Agentic RAG Self-Correction (LLM이 검색결과 평가 → 부족 시 쿼리 재작성, 최대 3회)

기술 스택:

- 오케스트레이션: LangGraph
- 벡터 DB: Qdrant
- 임베딩: Lucia Embedding Small
- 웹수집: Tavily API
- 콘텐츠 정제: Firecrawl
- LLM: Lucia 플랫폼
- 백엔드: FastAPI / 프론트엔드: Streamlit

평가 5트랙: 검색 품질, 페르소나 발산도, End-to-End 품질(RAGAS), 환각 검증, 시스템 성능

일정: 3주 (데이터 파이프라인 → 에이전틱 RAG/토론 → 통합/평가)

4조 — Travel Buddy in France (RAG 기반 여행 플래너)

주제: 프랑스 케이스로, 진정성 있고 개인화된 여행 추천을 제공하는 AI 어시스턴트

문제 정의:

- Tourist Trap 문제 (플랫폼들이 똑같은 인기 관광지만 추천)
- 개인화 부족 (분위기, 페이스 변화에 적응 못함)
- 광고/협찬으로 인한 비진정성

케이스 스터디: 프랑스 (전세계 관광객 7%)

기존 도구의 한계:

- 검색엔진: 사용자가 직접 필터링 필요
- 여행 사이트: 인기/스폰서 위주
- LLM 챗봇: 검색 없으면 일반적·구식 답변

제안 시스템:

- 프롬프팅 모듈로 챗봇이 여행 의도/예산/날짜/선호 장소 추출
- 정형 데이터(Google/Naver 지도) + 비정형 데이터(Reddit/Instagram 스크래핑)
- 에이전트가 문제 분해 후 일정 조립

기술 개선점:

1. Context-aware 개인화
2. 다양한 데이터 소스 + 구조화된 RAG 파이프라인
3. 에이전트 기반 플래닝 레이어

AWS 구성: S3, Lambda, EC2, API Gateway, CloudWatch, DynamoDB

평가: 3개 시스템 비교 (LLM only / 기본 RAG / 제안 RAG 에이전트), 100~200개 항목 + 40~60개 평가 질문

5조 — Hanyang University Agentic Map Assistant

발표자: Anna 외

주제: 한양대 캠퍼스 맵에 AI 어시스턴트를 통합한 인터랙티브 맵

문제 정의:

- 기본 맵은 위치 정보만 제공
- 실시간 정보(운영시간, 학식 메뉴, 날씨)는 여러 사이트에 흩어져 있음
- 신입생/교환학생/방문자에게 특히 어려움

기존 솔루션 한계:

- 정적 한양대 맵: 건물 이름만
- GPT/Gemini: 캠퍼스 실시간 정보 없음
- 한양 웹페이지: 접근 시간 소요

시스템 구조:

- 프론트엔드 ↔ FastAPI 백엔드
- Agent Orchestrator가 의도 분석 후 도구 선택
- **전문 도구들**:
  - Vector Search Tool (Supabase + pgvector)
  - Weather Tool
  - Facility Scraper Tool (운영시간/도서관 좌석)
  - Cafeteria Menu Tool

기술 스택:

- Python + FastAPI
- LangChain (에이전트 등록/툴 관리)
- LLM: GPT-5x 또는 Gemini 3x
- 임베딩: OpenAI 또는 Google text embeddings
- 인프라: Docker, ECR, ECS

최적화 전략: Hybrid Search, Query Expansion, Few-shot Prompting

6조 — Relay Mind (온보딩 컨텍스트 자동화)

주제: 프로젝트 기록을 실질적 온보딩 컨텍스트로 전환하는 AI 솔루션

문제 정의:

- 새 팀원 합류 시 노션/슬랙 정보가 늪이 됨
- 사수와 PO가 같은 질문에 매번 답변 (리소스 낭비)
- Gallup: 온보딩 만족 직원 단 12%, 평균 12개월 소요

차별점 (vs Slack AI / Notion AI):

- 단순 답변이 아닌 인수인계 패키지 자동 생성
- 단위가 다름: 의사결정 근거, 리스크, 다음 행동
- 결과물: 브리프 + 액션 플랜 (수동적 검색이 아닌 능동적)

핵심 기술 3가지:

1. **Agent Tools**: Google/Slack/Notion 연동
2. **RAG**: Vector RAG(빠른 검색) + Graph RAG(인과 관계 구조화)
3. **GPT 모델**: 맞춤형 온보딩 브리프 + 액션 플랜 생성

시스템 흐름:

- 슬랙(노이즈 제거) + 노션(계층 파싱) → Knowledge Engine → Graph DB + Vector DB → AI 온보딩 에이전트

산출물: 첫날 온보딩 브리프, 전임자 업무 캡슐, 1주일 액션 플랜, 온보딩 대시보드

인프라: AWS, React, Node.js, GPT, ChromaDB, Neo4j

평가: 학생 10~15명 + 실무자 3~5명 사용자 테스트, 5개 지표(이해도/완료시간/인지부하/신뢰도/사용 의향)

일정: 6주 (MVP 파일 업로드 방식)

7조 — YouthPath (청년 정책·취업 통합 에이전트)

주제: 청년의 정책/취업 정보 비대칭 해소하는 멀티에이전트 시스템

문제 정의:

1. 정보 장벽 - 행정 전문용어, LLM 데이터 컷오프
2. 실행 장벽 - 분절된 5~6개 플랫폼, 마감일 관리 리스크

핵심 기술 4가지:

- **LLM**: Lucia(한국어), Gemini 2.0 Flash(긴 문서), GPT-4 mini(평가)

- **RAG**: Static RAG + Dynamic RAG
- **MCP**: 실시간 API 호출
- **멀티에이전트**: 라우터 + 전문 에이전트

기존 서비스 vs 개선점:

- 정부 24: 추천 정책이 실제 대상 아닐 수 있음 → 정확한 자격 판단
- ChatGPT: 실시간 정보 약함 → RAG + MCP
- 잡코리아: 기업 맞춤 가이드 부족 → 사업보고서 기반
- 일반 캘린더: 자동 마감일 동기화 + 알림

4개 전문 에이전트:

1. **정책 에이전트** (온통청년 API + Static RAG)
2. **채용 에이전트** (워크넷/사람인 API + MCP)
3. **자소서 에이전트** (DART API + Dynamic RAG)
4. **캘린더 에이전트** (마감일 통합/알림)

평가: Jeopardy Q 방식, RAGAS 지표(사실 부합도, 검색 재현율/정밀도, 답변 관련성), GPT-4 mini Judge, 사용자 20명 테스트

일정: 5주 / 비용 약 60달러

8조 — Virtual User Research Platform (가상 한국 페르소나)

주제: 한국 합성 페르소나를 활용한 가상 유저 리서치 웹 플랫폼

문제 정의:

- 전통적 유저 리서치는 시간 소모/비용 부담/샘플 편향/스케일 한계

타겟 사용자: PM(A/B 테스트), UX 리서처(인구통계 커버리지), 스타트업(저렴한 검증)

5가지 핵심 컴포넌트:

1. **Persona DB**: NVIDIA Nemotron Persona Korea (700만개)
2. **LLM Pot**: Lucia Platform
3. **Chain of Thought** 프롬프팅
4. **RAG**: OpenSearch + KO embedding
5. **Agent**: LangChain + LangGraph (병렬 오케스트레이션)

경쟁 우위 (Synthetic Users / UserTesting AI 대비):

1. 한국 인구통계 (실제 통계 기반)
2. 데이터 소스 (62개 공공기관 데이터)
3. 지역/직업 필터링
4. 한국어 최적화 (KO-SRoBERTa)

프롬프트 3단계:

1. System Prompt (전체 페르소나 51개 필드)
2. Few-shot (2-3개 Q&A 페어)
3. Chain of Thought (왜 그렇게 느끼는지)

분석: KO-SRoBERTa 임베딩 → K-means 클러스터링 → LLM 클러스터 요약

기술 스택: React + D3.js (프론트), FastAPI (백), PostgreSQL (스토리지)

팀: 김나연(백엔드/인프라), 김나연(LLM/프롬프트), 이아누(RAG), 이한(에이전트), 차논안(프론트엔드/데모)

9조 — Multimodal Information Verification RAG

주제: 텍스트와 시각 정보의 불일치를 탐지하는 다중모달 정보 검증 시스템

문제 정의:

- 인터넷의 잘못된 정보 확산
- 설득력 있는 텍스트와 미묘하게 변조된 시각 자료 결합
- 뉴스/금융 등 사실이 중요한 영역에 적용

핵심 모델:

- **CLIP (OpenAI)**: 이미지와 텍스트를 같은 벡터 공간에 인코딩
- **DePlot (Google)**: 차트/그래프에서 정확한 수치 추출

기존 시스템과의 차이:

- 일반 RAG/요약: 입력을 신뢰한다는 가정
- 본 시스템: 모든 입력을 검증 대상으로 다룸 → 불일치 발견 시 상세 리포트 생성

3개 데이터베이스 아키텍처:

1. **Vector DB**: 텍스트/시각 특징 벡터 (CLIP)
2. **Knowledge Graph (Neptune)**: 인과관계, 출처, 콘텐츠 계보
3. **SQL DB**: 검증된 구조화 사실(공식 통계 등)

오케스트레이션: Amazon Bedrock Agents → LangChain + Claude 3.5 Sonnet

기술 스택: AWS SageMaker, OpenSearch, Neptune, FastAPI + React, ECS + Fargate

데이터: 금융 보고서 + 뉴스 미디어, 의도적 변조 테스트셋 생성

팀 5명: AI 모델러, 데이터 분석가, 데이터 엔지니어, 백엔드 개발자, 인프라 개발자

일정: 5주

10조 — Wafer Guard (반도체 결함 자동 탐지 MLOps)

주제: 반도체 결함을 자동 탐지하고 액션을 취하는 시스템

문제 정의 (4가지):

1. 결함이 너무 늦게 탐지됨 (재정 손실)
2. 결함 분석이 수동·비효율
3. 개별 엔지니어 전문성 의존 (지식 손실)
4. AI 시스템이 시간 경과 후 성능 저하/방치

비즈니스 임팩트: 작은 수율 개선도 큰 재정 이익 (삼성, SK하이닉스 투자)

기존 접근의 한계:

- 통계적 방법: 너무 단순
- 전통 ML: 이미지 처리 약함
- 딥러닝: 해석 어려움
- XAI: 시각화만 있고 설명 부족
- 대부분 파이프라인이 데이터 변화 대응 못함

데이터셋: WM811K (9개 결함 유형, 물리적 원인 명확)

시스템 아키텍처 (계층형):

- 프론트엔드 / API / **AI Core** / **MLOps** / 인프라

AI Core 기능:

- CNN 모델로 결함 분류
- Grad-CAM으로 결정 영역 식별
- LLM으로 사람이 읽을 수 있는 리포트
- RAG로 유사 과거 사례 검색
- 에이전트 워크플로우로 자동 의사결정

MLOps 핵심: 모델 버전 추적, 데이터 변화 탐지, 자동 재학습, 실시간 성능 모니터링

데모 시나리오 3가지:

1. 정상 운영 (즉시 분석)
2. 데이터 변화 탐지 → 자동 재학습
3. 성능 저하 시 이전 버전 롤백

일정: 4주

11조 — Debate Coach (AI 토론 코치)

주제: 개인화된 토론 훈련 도구 (음성 기반 상호작용)

문제 정의:

- 토론 연습 기회 부족 (시간/동료/강사 제약)
- 개별 피드백 어려움

핵심 차별점 (vs Agent for Debate):

- 사용자가 업로드한 문서에 RAG 기반 (환각 방지)
- 이기는 게 아니라 **가르치는 데** 설계됨
- 논리적 오류 탐지 + 피드백 리포트
- 음성 인터랙션 (목표: 5초 이하 응답 지연)

Phase 1 (Upload & Index):

1. React 웹앱에서 PDF 업로드
2. AWS Lambda + Python FastAPI → S3 저장
3. 토큰 청크 분할 → Lucia 임베딩 → FAISS 인덱스 → S3
4. Lucia가 입장 선택 + DynamoDB 세션 생성

Phase 2 (Live Debate Loop):

1. 마이크 입력 → S3
2. AWS Transcribe (음성 → 텍스트)
3. LangChain 에이전트가 Debator 역할
4. 임베딩 → FAISS 비교 → 상위 3개 청크
5. Lucia에 프롬프트 (페르소나 + 청크 + 메시지 + 오류 탐지 지시)
6. AWS Polly (TTS) → 사용자

Phase 3 (End Session):

1. Lucia가 전체 transcript 평가
2. JSON 저장
3. Gmail MCP로 recap 이메일 발송

3개 데이터셋:

1. RAG 평가용 70개 문서-쿼리 쌍 (15-20 문서)
2. 오류 탐지용 100개 인자 (60-70% valid)
3. 7개 토론 세션 + 사용자 설문

예산: $15

일정: 6주

12조 — Multi-Agent 투자 분석 리포팅 시스템

주제: 전문 애널리스트 수준의 분석을 개인 투자자에게 제공하는 멀티에이전트 시스템

문제 정의:

- 개인 투자자가 정형/비정형 데이터를 수동으로 연결해야 함
- 인지 부담 → 감정적 결정
- 기존 AI는 단일 모델 (편향, 검증 부재)

핵심 질문: "개인 투자자가 전문 애널리스트 수준의 분석을 받을 수 있는가?"

기존 한계:

- 전통 금융 앱: 데이터만 표시 (논리적 추론 없음)
- 범용 챗봇: 실시간 데이터 부족, 수치 계산 오류
- 단일 모델 → 단일 관점, 검증 부재

선행 연구 기반:

1. 금융 특화 모델이 일반 모델 능가
2. RAG로 정확도/투명성 향상
3. 멀티에이전트 토론으로 편향 감소

시스템 아키텍처 (LangChain 오케스트레이터):

- **RAG Agent**: 뉴스/공시/리포트 벡터 DB 검색
- **Quantitative Agent**: 외부 금융 API + Python 도구 (MCP) — 수치 오류 제거
- **Bullish Agent**: 성장 잠재력 분석 (낙관적)
- **Bearish Agent**: 리스크 중심 (비관적)
- **Judge Agent**: 4가지 기준(증거 신뢰도/수치 정확도/논리 일관성/리스크 커버리지)으로 통합 + 투자자 위험 선호도(1-10) 반영

기술 하이라이트:

1. Chain of Thought 프롬프팅
2. RAG 기반 출처 인용
3. MCP 기반 도구 통합 (수치 오류 방지)
4. Streamlit 실시간 대시보드

데이터: 정량(주가, PER, PBR — API), 정성(뉴스/공시/리포트 — RAG)

평가: 정량(Ground truth 비교), 정성(논리 일관성, 실제 리포트와 비교)

종합 비교

조	도메인	핵심 기술
1조	수강신청	LLM + RAG + 휴리스틱 스케줄러
2조	여행 일정	6단계 에이전트 + Live API
3조	페르소나 토론	Self-Correction RAG + Multi-Agent Debate
4조	여행(프랑스)	Community RAG + 에이전트 플래닝
5조	캠퍼스 맵	LangChain Agentic RAG + 4개 도구
6조	온보딩	Vector RAG + Graph RAG

조	도메인	핵심 기술
7조	청년 정책/취업	4 멀티에이전트 + Static/Dynamic RAG + MCP
8조	가상 유저 리서치	NVIDIA Persona + LangGraph 병렬
9조	정보 검증	CLIP + DePlot + Knowledge Graph
10조	반도체 결함	CNN + Grad-CAM + RAG + MLOps
11조	토론 코치	음성 RAG + LangChain + Gmail MCP
12조	투자 분석	Bullish/Bearish 토론 + Judge Agent + MCP

부록 — 7조 (YouthPath) 예상 Q&A

발표 Q&A 대비. 각 답변은 우리 시스템 구조(LUXIA + Gemini 2.0 Flash + GPT-4o-mini, Static/Dynamic RAG, MCP, LangGraph 4-Agent)에 맞춰 작성.

Q1. 3개의 LLM 모델을 쓰는 목적은? 단일 모델로는 안 되나요?

핵심: 각 모델이 잘하는 영역이 다르고, 단일 모델로 모든 요건을 만족시키면 비용·품질·객관성이 동시에 무너집니다.

답변:

- **LUXIA (한국어 특화)**: 사용자에게 노출되는 최종 응답 생성. 한국어 행정 용어와 자연스러운 어투에 최적화 — GPT 계열은 영어 기반이라 한국 정책 도메인에서 어색한 표현이 자주 나옴
- **Gemini 2.0 Flash (긴 문맥)**: 사업보고서·정책 공고처럼 수십 페이지짜리 비정형 문서 요약. 1M 토큰 컨텍스트 윈도우라 청크 분할 없이 통째로 처리 가능
- **GPT-4o-mini (평가)**: 우리 응답을 평가하는 LLM-as-Judge 역할. **응답 생성 모델로 평가하면 자기 편향이 생기므로** 반드시 다른 모델을 써야 함. 비용도 평가 용도엔 충분히 저렴

역할 분리의 이점:

1. 비용 — 짧은 응답엔 LUXIA, 긴 문서엔 Gemini Flash, 평가엔 mini → 단일 GPT-4 대비 훨씬 경제적
2. 품질 — 각 작업에 최적 모델
3. 객관성 — 평가 분리로 자기 편향 제거

Q2. 뉴스·정책 정보가 빠르게 변하는데 검증된 데이터를 어떻게 실시간 반영하나요?

핵심: 데이터 특성에 따라 Static RAG / Dynamic RAG / MCP 3단으로 나누어 처리합니다.

답변:

- **Dynamic RAG + MCP**: 채용 공고처럼 실시간 변경 데이터는 **캐싱 없이 매 요청마다 API 직접 호출** (워크넷, 사람인, DART). 호출 시점이 곧 최신성 보장
- **Static RAG 주기적 갱신**: 정책 본문은 자주 바뀌지 않지만, 매일/주간 단위 인덱스 재구축 + 폐지 정책은 메타데이터 플래그로 검색 결과에서 제외
- **출처 표시 의무화**: 모든 응답에 `[출처: 온통청년 API · 공고번호 · 갱신일]` 형식으로 사용자가 원본을 직접 검증 가능
- **last_updated 메타데이터**: 일정 기간 지난 정책엔 "최신 확인 필요" 플래그 표시
- **검증된 데이터만 사용**: 비공식 블로그·커뮤니티 데이터는 의도적으로 제외, 공공 API와 공시 자료만 사용 → 신뢰성 확보

Q3. 외부 API 지연·다운타임에 대한 Fallback 계획은?

핵심: Timeout + Retry + Graceful Degradation + Static RAG Fallback의 4단 방어 구조입니다.

답변:

1. **Timeout & Retry**: API 호출 5초 제한, 지수 백오프(1s → 2s → 4s)로 최대 3회 재시도
2. **Circuit Breaker**: 5분간 연속 실패율이 임계치를 넘으면 해당 API 일시 차단 → 무한 대기 방지
3. **Graceful Degradation**: 부분 실패 시 (예: Worknet 다운, Saramin 정상) 가능한 데이터로만 응답 + 명시적 안내 ("일부 채용 정보가 일시적으로 반영되지 않았습니다")
4. **Static RAG Fallback**: Dynamic RAG 실패 시 마지막 성공 시점에 캐싱된 Static 데이터로 fallback. 약간 outdated여도 사용자 경험은 유지
5. **명시적 신뢰도 표시**: "이 정보는 30분 전 데이터입니다" 같은 timestamp 표시로 사용자가 신뢰도 판단

Q4. Router Agent가 여러 에이전트가 동시에 필요한 질의를 어떻게 처리하나요? 병렬? 순차?

핵심: LangGraph가 의존성을 분석해 **독립적이면 병렬, 의존적이면 순차** 실행합니다.

답변:

- **병렬 실행**: 독립적인 작업
  - 예: "월세 지원 받으면서 할 수 있는 IT 인턴 알려줘" → Policy Agent + Job Agent를 `asyncio.gather`로 동시 호출
- **순차 실행**: 의존성 있는 작업
  - 예: "삼성전자 자소서 도와줘" → Job Agent로 회사 컨텍스트 확보 → Resume Agent로 자소서 가이드 생성
- **Router Agent 동작**:
  1. 사용자 질의 분석 → 필요한 에이전트 식별

2. 의존성 그래프 생성 (DAG)
3. 위상 정렬 후 가능한 단계는 병렬로
4. 결과는 Integrator Node에서 통합

- **LangGraph의 강점**: 상태 머신 기반이라 에이전트 간 데이터 전달과 분기·병합이 명시적으로 표현 가능

Q5. RAG-only를 추가 baseline으로 두면 에이전트 자체의 효과를 더 잘 측정할 수 있지 않을까요?

핵심: 좋은 지적입니다. **Ablation 형태로 다단 baseline**을 두는 것이 더 정확합니다.

답변:

현재 발표안은 단일 LLM vs YouthPath 2단 비교지만, 평가의 인과성을 분명히 하려면 다음과 같이 확장하겠습니다.

단계	시스템	측정 효과
Baseline 1	단일 LLM (Standard prompt)	LLM 자체 한계
Baseline 2	단일 LLM + RAG (no agents)	RAG 기여도
Baseline 3	RAG + Single Agent	멀티에이전트 분리 효과
Proposed	YouthPath (RAG + Multi-Agent + MCP)	전체 시스템 효과

이렇게 하면 RAG 기여도 vs 에이전트 분리 효과 vs MCP 효과를 분리 측정 가능합니다. RAGAS 4개 지표 (Faithfulness, Context Recall/Precision, Answer Relevance)를 각 단계에 동일하게 적용하면 ablation table이 자연스럽게 나옵니다.

Q6. 사용자가 어떤 input을 주고 어떤 output을 받는지?

Input (User Profile + Query):

- **프로필 (1회 입력)**: 전공, 경력, 나이, 거주지역, 소득 구간, 목표 직무
- **자연어 질의**: "월세 지원 받을 수 있는 거 있어?", "IT 신입 공고 추천", "삼성전자 자소서 어떻게 써?"
- **자소서 시**: 목표 회사 + 자소서 초안

Output (각 에이전트별):

에이전트	출력 형태
Policy	자격 매칭 정책 리스트 + 자격 충족 항목/부족 항목 + 신청 링크 + 출처
Job	필터링된 채용 공고 + 마감일 + 부합도 점수
Resume	Rubric 기반 피드백 (구체성/회사 핏/일관성 점수) + 회사 사업 키워드 기반 개선 제안

에이전트	출력 형태
Calendar	통합 마감일 캘린더 + 충돌 검사 + D-7/D-1 알림

모든 응답은 **출처 + 추천 이유**가 함께 표시됩니다. 예:

```
[추천] 청년월세지원
[출처] 온통청년 · 공고 2026-001 · 2026-04-15 갱신
[추천 이유]

- 만 27세 → 자격(만 19~34세) 충족

- 서울 거주 → 지역 요건 충족

- 소득 50% → 기준중위소득 60% 이하 충족
```

Q7. LUXIA, Gemini, GPT 혼합 사용 시 모델 간 데이터 전송 latency 최적화는?

핵심: **호출 경로 분리 + 병렬화 + 스트리밍**으로 사용자 체감 지연을 최소화합니다.

답변:

1. **역할 분리로 직렬 호출 최소화**:
   - LUXIA만 **사용자 실시간 응답 경로**에 위치
   - Gemini는 **백그라운드 사전 처리** (사업보고서 요약을 미리 캐싱)
   - GPT-4o-mini는 **오프라인 평가 단계**에서만 — 런타임 경로에 없음
   - 즉 사용자 한 번의 응답에 3개 모델을 직렬로 부르지 않음
2. **병렬화**: 독립적 모델 호출은 `asyncio.gather`로 동시 실행
3. **스트리밍**: LUXIA 토큰 스트리밍으로 첫 토큰 도달 시간(TTFT) 감소
4. **데이터 전송 최소화**: 모델 간 전달은 JSON 메타데이터로 작게 유지 (전체 청크 X, 요약·키워드만)
5. **캐싱**: 자주 묻는 질의 결과 + 자주 조회되는 정책 임베딩은 Chroma에 캐싱

Q8. 자격 검증에 추가 정보가 필요한 경우, 자동으로 사용자에게 추가 입력을 요청하나요?

핵심: 네, **Information Sufficiency Check 노드**가 LangGraph에 있어 자동 재질의합니다.

답변:

- **흐름**:
  1. Policy Agent가 매칭 시도 → 필수 필드 부족 감지 (예: 소득 구간 없음)
  2. Sufficiency Check 노드가 부족 항목 식별
  3. 사용자에게 자동 재질의: "청년월세지원 자격 확인을 위해 거주지역과 가구 소득을 알려주세요"

4. 사용자 응답 → 세션 상태 갱신 (DynamoDB) → 흐름 재개

- **세션 상태 유지**: Multi-turn 대화에서 이미 입력한 정보는 다시 묻지 않음
- **추정 모드**: 정보가 끝까지 부족하면 **가정을 명시한 일반적 답변** 제공 — "소득 정보가 없어 모든 구간 기준으로 안내드립니다" 식
- **점진적 입력**: 첫 사용 시 모든 정보를 한 번에 요구하지 않고, 필요할 때마다 점진적으로 수집 → UX 친화적

Q9. 구직자 입장에선 최신 정보가 더 중요한데, 오래된 정보는 DB에서 삭제되나요?

답변: 데이터 종류에 따라 다른 정책을 적용합니다.

데이터	정책
채용 공고 (Dynamic, 워크넷/사람인)	캐싱 안 함, 매번 API 호출 → 마감일 지난 공고 자동 제외
사업보고서 (DART)	최신 분기 보고서로 인덱스 자동 갱신, 이전 버전은 archived 플래그
정책 (Static, 온통청년)	폐지된 정책은 `status: deprecated` 플래그 → 검색 결과에서 자동 제외, 단 과거 수령자 문의 대비 데이터 자체는 보존
벡터 인덱스	30일 미갱신 시 자동 재구축 트리거

메타데이터: 모든 문서에 `last_updated`, `effective_date`, `expiry_date` 부여 → "오래된 정보입니다" 경고를 사용자에게 명시적으로 표시.

삭제 vs 비활성화: 삭제하지 않고 비활성화(soft delete)함. 이유: (1) 사용자가 과거 신청한 정책 추적 가능, (2) 평가용 ground truth로 재활용 가능.

Q10. Static RAG와 Dynamic RAG 각각의 query/output 예시는?

Static RAG — "변하지 않는 정책 본문"

- **Query**: "청년월세지원 자격 요건이 뭐야?"
- **Data Source**: KOGL 정책 DB (Chroma 벡터 인덱스, 사전 임베딩됨)
- **Process**: 사용자 질의 임베딩 → 벡터 유사도 검색 → 관련 정책 문서 청크 retrieval
- **Output**: "만 19~34세, 기준중위소득 60% 이하 무주택자, 임차 보증금 5천만원 이하... [출처: 청년월세지원 시행공고 2026]"

Dynamic RAG — "실시간 변하는 데이터"

- **Query**: "IT 분야 신입 채용 공고 추천해줘"
- **Data Source**: 워크넷 + 사람인 API (실시간 호출)

- **Process**: MCP 통한 API 호출 → 직무·지역·경력 필터링 → 응답 가공
- **Output**: "오늘 등록된 IT 신입 공고 5개: ① OO회사 백엔드 (마감 D-7), ② △△회사 데이터 (마감 D-3)..."

Hybrid (Resume Agent) — "긴 문서 + 실시간"

- **Query**: "삼성전자 자소서 가이드"
- **Data Source**: DART API (사업보고서 PDF, 실시간) + Saramin (채용공고)
- **Process**: DART에서 최신 사업보고서 fetch → Gemini 2.0 Flash로 요약 (긴 문맥) → Saramin 채용 공고와 매칭
- **Output**: "삼성전자 2024 사업보고서 기준 — 주요 사업: AI 반도체/HBM; R&D: 35% 증가; 리스크: 글로벌 경쟁 → 자소서에 본인의 'AI 관련 프로젝트' 또는 '반도체 도메인 경험'을 연결하세요"

Q11. 자소서 가이드가 누구에게나 적용 가능한 일반론으로 흐를 위험을 어떻게 줄이나요?

핵심: **회사 사업보고서 + 채용 공고 + 사용자 이력**의 3중 결합으로 일반론을 차단합니다.

답변:

1. **회사별 DART 사업보고서 기반 구체화**:
   - 일반화된 "성장하는 회사" 표현 금지
   - 대신 "이 회사는 2024년 AI 반도체 R&D를 35% 증액", "주력 사업: HBM·파운드리"같이 **수치와 사실 기반**으로 가이드 생성
2. **Saramin 채용 공고와 자격 요건 매칭**:
   - 공고의 우대 사항 vs 사용자 이력 비교 → "공고는 'Python·SQL' 우대, 본인 이력 매칭 ✓ → 강조 권장"같이 **항목별 매칭** 제시
3. **Rubric 기반 평가**:
   - 4개 축으로 점수화: ① 회사 핏(specificity to company), ② 직무 핏(role match), ③ 본인 경험과의 연결성, ④ 일반론 비율
   - 일반론 비율이 임계치 초과 시 자동 재생성 트리거
4. **일반화 표현 자동 감지**:
   - "도전적인 인재", "성장하는 회사", "글로벌" 같은 양적·질적 근거 없는 표현 감지 → 경고 또는 재생성
5. **사용자 이력서 + 공고 + 회사 사업의 교집합 강조**:
   - 가이드의 모든 추천 포인트는 반드시 위 3개 데이터 중 2개 이상에서 근거 제시 가능해야 함

출력 예시 (일반론 X):

- (X) "이 회사의 비전에 부합하는 인재상을 어필하세요"
- (O) "삼성전자는 2024 사업보고서에서 'AI·6G' 투자를 명시했고, 본인의 '경량화 모델 프로젝트' 경험은 직접 연결됩니다 → 이 프로젝트의 정량적 성과(파라미터 수, 추론 속도)를 1문단으로 강조하세요"

답변 시 강조할 공통 메시지

질문이 어떻게 들어오든 다음 메시지로 수렴시키면 좋습니다.

1. **데이터 특성에 따라 분리 처리** (Static RAG / Dynamic RAG / MCP)
2. **단일 모델이 아닌 역할 분리** (LUXIA / Gemini / GPT — 각자 잘하는 일)
3. **출처 + 추천 이유 명시** (할루시네이션 차단)
4. **End-to-End 라이프사이클** (정책 + 채용 + 자소서 + 캘린더)
5. **현실적 fallback 설계** (외부 API 실패 시 대응)